

8Byte.ai DevOps Engineer Technical Assignment

Deliverable: Challenges Faced, Root Causes & Resolutions

Candidate: Sarthak Shirbhate	Target Role: Senior DevOps / SRE Engineer
Organization: 8Byte.ai	Date: August 2026
Repository: github.com/sarthak24shirbhate/8byte-devops-assignment	Status: Complete & Validated

Challenge 1: Circular Dependency in Security-Group-to-Security-Group Rule Definitions

Problem:	Defining bidirectional least-privilege security group rules where ALB allows egress strictly to ECS, and ECS allows ingress strictly from ALB, triggered cyclic dependency evaluation errors in Terraform's DAG.
Root Cause:	Inline ingress/egress blocks defined inside <code>aws_security_group</code> resources create mutual cross-dependencies preventing either security group from determining its ID first.
Resolution:	Decoupled security groups from rules by utilizing standalone <code>aws_security_group_rule</code> resources. Security groups are provisioned first, followed by rules referencing peer IDs.
Validation:	Executed <code>terraform validate</code> and <code>terraform plan</code> ; DAG constructed clean acyclic graph.
Key Learning:	Always separate security group declarations from cross-referencing rules when implementing zero-trust perimeter boundaries in IaC.

Challenge 2: Chicken-and-Egg Dilemma with Remote State Backend Creation

Problem:	Terraform S3 backend configuration requires the remote S3 bucket and DynamoDB lock table to exist before <code>terraform init</code> can run, blocking automated single-command provisioning.
Root Cause:	Backend initialization occurs before any root module resources are evaluated or provisioned.
Resolution:	Constructed an isolated <code>terraform/bootstrap/</code> module that provisions the encrypted, versioned S3 bucket and DynamoDB table independently prior to root state initialization.
Validation:	Ran <code>terraform init -backend=false</code> and validated bootstrap outputs with backend migration.
Key Learning:	Decouple state storage lifecycles from application infrastructure to maintain reproducible GitOps workflows.

Challenge 3: SQLAlchemy Schema Initialization during FastAPI TestClient Execution

Problem:	Executing <code>pytest</code> on API endpoints failed with <code>OperationalError: no such table: items</code> .
Root Cause:	FastAPI lifespan handlers are not triggered by default when <code>TestClient</code> is instantiated outside of an ASGI context manager.
Resolution:	Implemented <code>tests/conftest.py</code> with an <code>autouse</code> session fixture (<code>setup_test_db</code>) and client context fixture to manage table creation and teardown independently.

Validation:	Re-ran pytest tests/ -v; all 6 unit and integration test cases passed with 100% success rate.
Key Learning:	Test suites must manage test database schemas via standardized test fixtures rather than relying on application startup lifecycle hooks.

Challenge 4: AWS IAM OpenID Connect (OIDC) Scoping for Keyless GitHub Actions Authentication

Problem:	Storing long-lived AWS Access Keys in GitHub Secrets introduces security credential leakage vulnerabilities.
Root Cause:	Traditional CI/CD configurations rely on static IAM user credentials rather than federated identity tokens.
Resolution:	Implemented modules/oidc/ provisioning an aws_iam_openid_connect_provider and an IAM role with trust policy strictly scoped to repo:sarthak24shirbhate/8byte-devops-assignment:* and audience sts.amazonaws.com.
Validation:	Validated IAM trust policy and verified ephemeral STS token exchange mechanism.
Key Learning:	Ephemeral STS tokens via OIDC completely eliminate the risk of leaked permanent cloud credentials.

Challenge 5: Secure RDS Master Password Injection into ECS Tasks without Plaintext Leakage

Problem:	Application containers require database credentials at launch, but embedding passwords in Git, env files, or .tfvars violates security policies.
Root Cause:	Passing database passwords via standard environment variables exposes them in task definitions and state outputs.
Resolution:	Generated credentials using random_password, stored payload in AWS Secrets Manager, and configured ECS task definition secrets array (valueFrom) with IAM Task Execution role permissions.
Validation:	Confirmed task definition JSON references Secret ARN and that terraform outputs mask passwords.
Key Learning:	Using AWS native IAM execution role resolution ensures credentials are retrieved in-memory at launch without persistent configuration exposure.

Challenge 6: CloudWatch Dashboard Metric Dimension Resolution Across Modular Outputs

Problem:	CloudWatch ApplicationELB metrics displayed empty widgets because they require ARN Suffix strings rather than full ARNs.
Root Cause:	CloudWatch namespace dimensions for ELB strictly expect app// and targetgroup// strings.
Resolution:	Exported aws_lb.main.arn_suffix and aws_lb_target_group.app.arn_suffix from modules/alb/outputs.tf and wired them into modules/monitoring/main.tf widgets.
Validation:	Validated CloudWatch Dashboard JSON against AWS metric dimension schemas.

Key Learning:	Always inspect provider-specific metric dimension formats when constructing IaC monitoring dashboards.
----------------------	--
